

Log and failure triage

When make sim fails, the log is your first stop, not a random rewrite of the RTL

Tail, grep, then classify

- Start with the last lines, tail shows where the run stopped
- Grep for error and warn, case-insensitive, so you do not miss mixed case
- Ask: is this expect-got in the testbench
- Classification decides whether you fix RTL, fix env, or chase timing

Browser lab

Digital Design and Verification Platform

HomeToolsLog triage

Read Make and sim log snippets — classify each as a real fail, an env problem, or a flake.

Starter example: Open case 1 (TB mismatch) — that is a real fail. Scan for command not found (env) vs intermittent timeout (flake).

Load starter example

Challenges 0 / 22 cleared

Quiz: buckets: Three triage buckets: fail, env, and? Answer: flake

Answer

Show hintCheckNextIdle

Quiz: bucketsQuiz: envQuiz: failCase: mismatchCase: missing toolCase: timeoutCase: syntaxCase: licenseCase: X-propCase: diskCase: parallelCase: includeCase: assertCase: cooohCase: xaze CDCQuiz: first errorQuiz: flake ticketSix correctAll twelveQuiz: 127Quiz: makeStarter case

Buckets

fail

Deterministic DUT/TB/assert/compile error in your sources.

env

Tools, PATH, license, disk, missing files/submodules.

flake

Intermittent: races, timeouts, parallel collisions — re-run differs.

Log case

123456789101112

Case 1/32 · 0 classified · 0 correct

TB expect mismatch

make sin · tb_alu

```
0 mlec nio Tbtb_alu
vvp build/tb_alu.vvp
[tb] time=120 ns expect y=0 h2a got 0'h00
ERROR: mismatch at vector 4
make: *** [sin] error 1
```

Look for ERROR: mismatch

Look for expect/got

Classify

failenvflake

Pick a bucket

Classify the log, then see why.

History

0 starter: case 1 is a deterministic TB fail

Cheat sheet

Signal	Lean toward
expect/got, assert, syntax error	fail
command not found, license, disk, ModuleNotFound	env
timeout + prior pass, -j collision, 1/N rare	flake

Read the first actionable error, not only the final Make status.

Flakes still got tickets — triage ≠ ignore.

Env fixes are often one-line PATH / init / quota.

Log triage lab

learn_unix — log triage

Real shell practice

```
wsl - module26-log-triage/examples/logs

# Track A - real Unix
# real shell session (Track A)

$ pwd
/mnt/d/proj/designs/learning/courses/learn_unix/module26-log-triage/examples/logs

$ ls -la
total 4
drwxrwxrwx 1 yongfu yongfu 4096 Jul 18 16:34 .
drwxrwxrwx 1 yongfu yongfu 4096 Jul 20 00:49 ..
-rwxrwxrwx 1 yongfu yongfu 1105 Jul 20 09:17 README.md
-rwxrwxrwx 1 yongfu yongfu  203 Jul 20 09:17 commands_to_try.txt
-rwxrwxrwx 1 yongfu yongfu  401 Jul 18 16:34 sample.log

$ wc -l sample.log
11 sample.log

$ tail -5 sample.log
[ERROR] Assertion failed at line 42: expected 0x0, got 0x1
[WARN] Timeout threshold approaching
[INFO] Test run finished
[ERROR] 1 test failed
[INFO] End of log

$ grep -i error sample.log
[ERROR] Assertion failed at line 42: expected 0x0, got 0x1
[ERROR] 1 test failed

$ grep -i warn sample.log
[WARN] Unused signal 'debug_q' in block top
[WARN] Timeout threshold approaching

$ grep -n ERROR sample.log
7:[ERROR] Assertion failed at line 42: expected 0x0, got 0x1
10:[ERROR] 1 test failed
```

Real shell — log triage

Real shell practice — try these

```
# pwd — print working directory (where am I?)  
pwd
```

```
# ls -la — list all entries, long format (what is here?)  
ls -la
```

```
# wc -l sample.log — count lines in the sample log  
wc -l sample.log
```

```
# tail -5 sample.log — show the last five lines (how did it end?)  
tail -5 sample.log
```

```
# grep -i error sample.log — find error lines, case-insensitive  
grep -i error sample.log
```

```
# grep -i warn sample.log — find warn lines, case-insensitive
```

Pitfalls to watch

- Do not rewrite RTL when the log says command not found
- Do not ignore warn lines that foreshadow a timeout
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, classify a few fail, env, and flake cases after the starter
- On the real shell, triage the sample log with tail and grep
- When you are ready, take the short quiz, then continue to env-file literacy